

Integración de Login Azure con RepositoryUta

Guía genérica para nuevos frontends o clientes web que consumirán los servicios de autenticación Azure.

1. Objetivo

Este documento indica, de forma independiente a cualquier frontend existente, qué debe parametrizarse en RepositoryUta y cuál es el orden exacto de consumo de servicios para autenticar un usuario con Microsoft Azure / Office 365.

El objetivo es que un programador pueda implementar un login Azure desde cero sabiendo: tablas requeridas, parámetros obligatorios, endpoints, orden del flujo y forma correcta de recibir la notificación SignalR.

Tablas a parametrizar	Parámetros necesarios
Llaves de seguridad del flujo	Servicios a consumir
Orden completo del flujo	Recepción de SignalR
Código de referencia	Errores frecuentes

2. Tablas que deben registrarse antes del login

RepositoryUta solo permite generar URL Azure para aplicaciones registradas. Antes de que un frontend pueda consumir el login, deben existir registros válidos en las siguientes tablas.

2.1 Tabla auth.tbl_Applications

Registra la aplicación cliente. El campo más importante para el frontend es `clientId`. Este valor es público y será enviado por el frontend en la URL Azure y en la conexión SignalR.

Campo	Requerido	Uso	Ejemplo
Id	Sí	Identificador interno GUID.	NEWID()
Name	Sí	Nombre descriptivo de la aplicación.	Portal Nuevo

Campo	Requerido	Uso	Ejemplo
ClientId	Sí	Identificador público que usará el frontend.	portal-nuevo-web
ClientSecretHash	Sí	Hash del secreto para flujos backend/legacy. No se entrega al frontend público.	HASH_GENERADO
Description	No	Descripción funcional.	Frontend nuevo con login Azure
IsActive	Sí	Debe estar en 1 .	1
IsDeleted	Sí	Debe estar en 0 .	0

Nota para frontend: el `ClientSecret` no debe incluirse en aplicaciones web públicas. El frontend solo necesita conocer el `ClientId` .

2.2 Tabla `auth.tbl_NotificationSubscriptions`

Registra la suscripción que permite a RepositoryUta notificar el resultado del login Azure. Para que un navegador reciba el evento por SignalR, el campo `NotificationType` debe ser `websocket` O `both` .

Campo	Requerido	Uso	Valor recomendado
Id	Sí	Identificador de suscripción.	NEWID()
ApplicationId	Sí	Debe apuntar a <code>auth.tbl_Applications.Id</code> .	@ApplicationId
EventType	Sí	Evento que se desea recibir.	Login
WebhookUrl	No	Solo si se usa webhook o both.	NULL para solo SignalR
SecretKey	No	Clave HMAC para webhook.	NULL para solo SignalR
NotificationType	Sí	Canal de notificación.	websocket
WebSocketGroupName	No	Referencia del grupo de app.	app_portal-nuevo-web

Campo	Requerido	Uso	Valor recomendado
RequireAuthentication	No	Para login inicial suele ser anónimo.	0
IsActive	Sí	Debe estar activa.	1

2.3 Validación rápida de registros

Para validar la parametrización, confirmar que el `clientId` exista activo y no eliminado en `auth.tbl_Applications` . Luego confirmar que la aplicación tenga una suscripción activa en `auth.tbl_NotificationSubscriptions` CON `EventType = "Login"` y `NotificationType = "websocket"` O `"both"` .

3. Parámetros que necesita crear o manejar el frontend

Parámetro	Quién lo crea	Dónde se usa	Obligatorio
<code>authBaseUrl</code>	Configuración del frontend	Base de endpoints REST.	Sí
<code>notificationHubUrl</code>	Configuración del frontend	Conexión SignalR.	Sí
<code>clientId</code>	RepositoryUta, registrado en <code>auth.tbl_Applications</code>	<code>/api/auth/azure/url</code> y <code>/notificationHub</code> .	Sí
<code>browserId</code>	Frontend	Identifica el navegador que debe recibir la notificación.	Sí
<code>codeVerifier</code>	Frontend	Se usa al canjear <code>deliveryCode</code> .	Sí
<code>codeChallenge</code>	Frontend, calculado desde <code>codeVerifier</code>	Se envía a <code>/api/auth/azure/url</code> .	Sí
<code>deliveryCode</code>	RepositoryUta	Llega en <code>LoginNotification</code> ; se envía a <code>/api/auth/azure/exchange</code> .	Sí

3.1 Valores de ejemplo

```
authBaseUrl = "https://repositoryuta.uta.edu.ec"
notificationHubUrl = "https://repositoryuta.uta.edu.ec/notificationHub"
clientId = "portal-nuevo-web"
browserId = "uuid-generado-por-el-frontend"
```

```
codeVerifier = "random-base64url-generado-en-la-pestana"  
codeChallenge = "base64url(SHA256(codeVerifier))"
```

4. Llaves de seguridad y no broadcast

El flujo actualizado evita enviar el resultado de login a todos los clientes conectados al hub. Para eso usa una combinación de `clientId`, `browserId`, `state`, `codeChallenge`, `codeVerifier` y `deliveryCode`.

Elemento	Quién lo crea	Viaja por red	Función de seguridad
<code>clientId</code>	RepositoryUta	Sí	Identifica la aplicación cliente autorizada en <code>auth.tbl_Applications</code> .
<code>browserId</code>	Frontend	Sí	Llave de enrutamiento para que RepositoryUta envíe la notificación solo al grupo <code>browser_{browserId}</code> .
<code>state</code>	RepositoryUta	Sí, por OAuth	Transporta y valida <code>stateId</code> , <code>clientId</code> , <code>browserId</code> y <code>codeChallenge</code> . Se valida contra caché anti replay.
<code>codeVerifier</code>	Frontend	Solo al canjear	Llave privada temporal de la pestaña. No debe guardarse en base ni enviarse al iniciar el login.
<code>codeChallenge</code>	Frontend	Sí	Hash PKCE del <code>codeVerifier</code> . RepositoryUta lo guarda ligado al flujo para validar el canje.
<code>deliveryCode</code>	RepositoryUta	Sí, por SignalR	Código de un solo uso. Sustituye el envío directo de tokens por el hub.

Cuando `browserId` está presente, RepositoryUta envía `LoginNotification` al grupo `browser_{browserId}`. Solo usa `app_{clientId}` como ruta amplia si no existe `browserId`. Por seguridad, un frontend nuevo siempre debe enviar `browserId`.

El valor `state` viaja codificado en Base64 dentro del flujo OAuth. No debe tratarse como cifrado. La seguridad sensible del canje se basa en PKCE: solo la pestaña que conserva el `codeVerifier` puede canjear el `deliveryCode` por tokens.

5. Servicios que consume el frontend

Orden	Método	Endpoint	Parámetros	Respuesta importante
1	WS	/notificationHub? clientId= {clientId}&browserId= {browserId}	Query: clientId , browserId	Conexión SignalR activa.
2	SignalR	JoinApplicationGroup	clientId , userId opcional o null	Conexión asociada a app_{clientId} .
3	SignalR	JoinBrowserGroup	clientId , browserId	Conexión asociada a browser_{browserId} .
4	GET	/api/auth/azure/url	clientId , browserId , codeChallenge	url de Microsoft y state .
5	GET	/api/auth/azure/callback	code , state	Lo invoca Microsoft. El frontend no llama este endpoint manualmente.
6	SignalR	LoginNotification	Evento recibido por el frontend.	deliveryCode y datos básicos del usuario.
7	POST	/api/auth/azure/exchange	deliveryCode , codeVerifier	accessToken , refreshToken .
8	GET	/api/auth/me	Header Authorization: Bearer accessToken	Datos de usuario, roles, permisos y menús.

5.1 Servicio para obtener URL Azure

GET {authBaseUrl}/api/auth/azure/url?clientId=portal-nuevo-web&browserId={browserId}&codeChallenge

Respuesta esperada

```
{
  "success": true,
  "data": {
    "url": "https://login.microsoftonline.com/...",
    "state": "base64-state",
    "clientId": "portal-nuevo-web",
    "browserId": "uuid-del-navegador"
```

```
}
}
```

5.2 Servicio para canjear deliveryCode

POST {authBaseUrl}/api/auth/azure/exchange
Content-Type: application/json

```
{
  "deliveryCode": "codigo-recibido-por-signalr",
  "codeVerifier": "verifier-generado-por-la-pestana"
}
```

Respuesta esperada

```
{
  "success": true,
  "data": {
    "accessToken": "jwt",
    "refreshToken": "refresh-token"
  }
}
```

5.3 Servicio para obtener usuario actual

GET {authBaseUrl}/api/auth/me
Authorization: Bearer {accessToken}

6. Orden completo del flujo de login

1. El usuario presiona el botón **Iniciar sesión con Azure**.
2. El frontend lee o crea `browserId` . Debe persistirlo en almacenamiento local del navegador.
3. El frontend genera `codeVerifier` . Este valor debe quedarse solo en memoria de la pestaña.
4. El frontend calcula `codeChallenge` usando SHA-256 y Base64URL.
5. El frontend crea la conexión SignalR a `/notificationHub?clientId={clientId}&browserId={browserId}` .
6. El frontend registra el listener del evento `LoginNotification` .
7. El frontend inicia la conexión SignalR con `connection.start()` .
8. El frontend invoca `JoinApplicationGroup(clientId, null)` .

9. El frontend invoca `JoinBrowserGroup(clientId, browserId)`.
 10. El frontend consume `GET /api/auth/azure/url` enviando `clientId`, `browserId` y `codeChallenge`.
 11. `RepositoryUta` valida que `clientId` exista activo en `auth.tbl_Applications`.
 12. `RepositoryUta` responde la URL de Microsoft.
 13. El frontend abre la URL de Microsoft en popup o redirección.
 14. El usuario se autentica en Microsoft.
 15. Microsoft redirige a `/api/auth/azure/callback` CON `code` y `state`.
 16. `RepositoryUta` valida `state`, intercambia el código con Microsoft y crea tokens internos.
 17. `RepositoryUta` crea un `deliveryCode` de un solo uso.
 18. `RepositoryUta` busca una suscripción activa en `auth.tbl_NotificationSubscriptions` para `EventType = Login`.
 19. `RepositoryUta` envía `LoginNotification` únicamente al grupo `browser_{browserId}` si el frontend envió `browserId`.
 20. El frontend recibe `deliveryCode`.
 21. El frontend consume `POST /api/auth/azure/exchange` enviando `deliveryCode` y `codeVerifier`.
 22. `RepositoryUta` valida que el `codeVerifier` corresponda al `codeChallenge` original.
 23. `RepositoryUta` responde `accessToken` y `refreshToken`.
 24. El frontend guarda los tokens según su estrategia de sesión.
 25. El frontend consume `GET /api/auth/me` para cargar usuario, roles y permisos.
 26. El frontend redirige al usuario a su pantalla inicial autenticada.
-

7. Cómo receptar la notificación SignalR

6.1 Conexión al hub

```
notificationHubUrl = "{authBaseUrl}/notificationHub"  
connectionUrl = "{notificationHubUrl}?clientId={clientId}&browserId={browserId}"
```

6.2 Métodos que se invocan luego de conectar

```
await connection.start();  
await connection.invoke("JoinApplicationGroup", clientId, null);  
await connection.invoke("JoinBrowserGroup", clientId, browserId);
```

6.3 Evento que debe escuchar el frontend

```
connection.on("LoginNotification", async (message) => {
  if (message.eventType !== "Login") return;

  const deliveryCode = message.deliveryCode;
  // Canjear deliveryCode con /api/auth/azure/exchange
});
```

6.4 Payload esperado

```
{
  "eventType": "Login",
  "timestamp": "2026-07-24T10:00:00",
  "context": {
    "initiatingApplication": "portal-nuevo-web",
    "loginSource": "Office365",
    "sessionScope": "specific",
    "notificationType": "hybrid"
  },
  "data": {
    "userId": "guid",
    "email": "usuario@uta.edu.ec",
    "displayName": "Nombre Usuario",
    "loginType": "Office365",
    "ipAddress": "127.0.0.1",
    "roles": ["Rol1"],
    "permissions": ["/ruta"]
  },
  "pair": null,
  "deliveryCode": "codigo-de-un-solo-uso"
}
```

El frontend debe completar el login usando `deliveryCode` . No debe depender de `pair` , porque en el flujo seguro los tokens reales no viajan por SignalR.

8. Código de referencia para un frontend nuevo

Este ejemplo es genérico. Puede adaptarse a React, Angular, Vue o JavaScript puro. Requiere la librería `@microsoft/signalr` .

```
import * as signalR from "@microsoft/signalr";

const authBaseUrl = "https://repositoryuta.uta.edu.ec";
const notificationHubUrl = `${authBaseUrl}/notificationHub`;
const clientId = "portal-nuevo-web";
```



```
function getBrowserId() {
  const key = "repositoryuta-browser-id";
  let browserId = localStorage.getItem(key);

  if (!browserId) {
    browserId = crypto.randomUUID();
    localStorage.setItem(key, browserId);
  }

  return browserId;
}

function base64UrlEncode(bytes) {
  let binary = "";
  for (const byte of bytes) {
    binary += String.fromCharCode(byte);
  }
  return btoa(binary)
    .replace(/\+/g, "-")
    .replace(/\//g, "_")
    .replace(/=+$/, "");
}

function generateCodeVerifier() {
  const bytes = new Uint8Array(32);
  crypto.getRandomValues(bytes);
  return base64UrlEncode(bytes);
}

async function computeCodeChallenge(codeVerifier) {
  const data = new TextEncoder().encode(codeVerifier);
  const digest = await crypto.subtle.digest("SHA-256", data);
  return base64UrlEncode(new Uint8Array(digest));
}

async function loginWithAzure() {
  const browserId = getBrowserId();
  const codeVerifier = generateCodeVerifier();
  const codeChallenge = await computeCodeChallenge(codeVerifier);

  const connection = new signalR.HubConnectionBuilder()
    .withUrl(`${notificationHubUrl}?clientId=${encodeURIComponent(clientId)}&browserId=${encodeURIComponent(browserId)}`)
    .withAutomaticReconnect()
    .build();

  connection.on("LoginNotification", async (message) => {
    if (!message || message.eventType !== "Login") return;

    if (!message.deliveryCode) {
      throw new Error("No llegó deliveryCode en LoginNotification.");
    }

    const exchangeResponse = await fetch(`${authBaseUrl}/api/auth/azure/exchange`, {
      method: "POST",
    });
  });
}
```

```
headers: { "Content-Type": "application/json" },
body: JSON.stringify({
  deliveryCode: message.deliveryCode,
  codeVerifier: codeVerifier
})
});

const exchangePayload = await exchangeResponse.json();

if (!exchangeResponse.ok || !exchangePayload.success) {
  throw new Error(exchangePayload.message || "No se pudo canjear deliveryCode.");
}

const accessToken = exchangePayload.data.accessToken;
const refreshToken = exchangePayload.data.refreshToken;

const meResponse = await fetch(`${authBaseUrl}/api/auth/me`, {
  headers: {
    Authorization: `Bearer ${accessToken}`
  }
});

const mePayload = await meResponse.json();

if (!meResponse.ok || !mePayload.success) {
  throw new Error(mePayload.message || "No se pudo obtener usuario actual.");
}

localStorage.setItem("accessToken", accessToken);
localStorage.setItem("refreshToken", refreshToken);
localStorage.setItem("userSession", JSON.stringify(mePayload.data));

await connection.stop();
window.location.href = "/";
});

await connection.start();
await connection.invoke("JoinApplicationGroup", clientId, null);
await connection.invoke("JoinBrowserGroup", clientId, browserId);

const query = new URLSearchParams({
  clientId,
  browserId,
  codeChallenge
});

const azureUrlResponse = await fetch(`${authBaseUrl}/api/auth/azure/url?${query.toString()}`);
const azureUrlPayload = await azureUrlResponse.json();

if (!azureUrlResponse.ok || !azureUrlPayload.success) {
  throw new Error(azureUrlPayload.message || "No se pudo obtener URL de Azure.");
}

const popup = window.open(
  azureUrlPayload.data.url,
```

```
"azure-login",  
"width=600,height=720,left=200,top=100"  
);  
  
if (!popup) {  
    throw new Error("El navegador bloqueó el popup de Azure.");  
}  
}
```

8.1 Orden resumido en pseudocódigo

```
browserId = getOrCreateBrowserId()  
codeVerifier = generateRandomPkceVerifier()  
codeChallenge = sha256Base64Url(codeVerifier)  
  
connection = connectSignalR(notificationHubUrl, clientId, browserId)  
listen LoginNotification  
start connection  
invoke JoinApplicationGroup(clientId, null)  
invoke JoinBrowserGroup(clientId, browserId)  
  
azureUrl = GET /api/auth/azure/url(clientId, browserId, codeChallenge)  
openPopup(azureUrl)  
  
on LoginNotification:  
    tokens = POST /api/auth/azure/exchange(deliveryCode, codeVerifier)  
    user = GET /api/auth/me(accessToken)  
    saveSession(tokens, user)  
    redirect()
```

9. Errores frecuentes y validaciones

Situación	Causa probable	Validación
Aplicación cliente no autorizada	No existe ClientId activo en auth.tbl_Applications .	Revisar ClientId , IsActive = 1 , IsDeleted = 0 .
No llega LoginNotification	No existe suscripción activa o NotificationType no es websocket / both .	Revisar auth.tbl_NotificationSubscriptions .
La notificación no llega al navegador correcto	El browserId usado en SignalR no coincide con el enviado a /api/auth/azure/url .	Loguear ambos valores y confirmar igualdad exacta.

Situación	Causa probable	Validación
<code>deliveryCode</code> inválido o expirado	El código ya fue usado, expiró o se perdió el <code>codeVerifier</code> .	Reiniciar el flujo de login Azure.
Popup bloqueado	El popup no se abrió desde una acción directa del usuario.	Abrir Microsoft desde el click del botón de login.
Error CORS o SignalR no conecta	Origen del frontend no permitido o URL de hub incorrecta.	Validar CORS, HTTPS, proxy y <code>notificationHubUrl</code> .

9.1 Checklist final para el programador

1. Solicitar `clientId` registrado en RepositoryUta.
2. Confirmar que existe suscripción `Login` con `NotificationType = websocket` O `both` .
3. Configurar `authBaseUrl` y `notificationHubUrl` .
4. Implementar generación de `browserId` .
5. Implementar PKCE: `codeVerifier` y `codeChallenge` .
6. Conectar SignalR antes de abrir Microsoft.
7. Invocar `JoinBrowserGroup` con el mismo `browserId` .
8. Solicitar URL Azure.
9. Recibir `LoginNotification` .
10. Canjear `deliveryCode` .
11. Consultar `/api/auth/me` .
12. Guardar sesión y continuar al sistema.